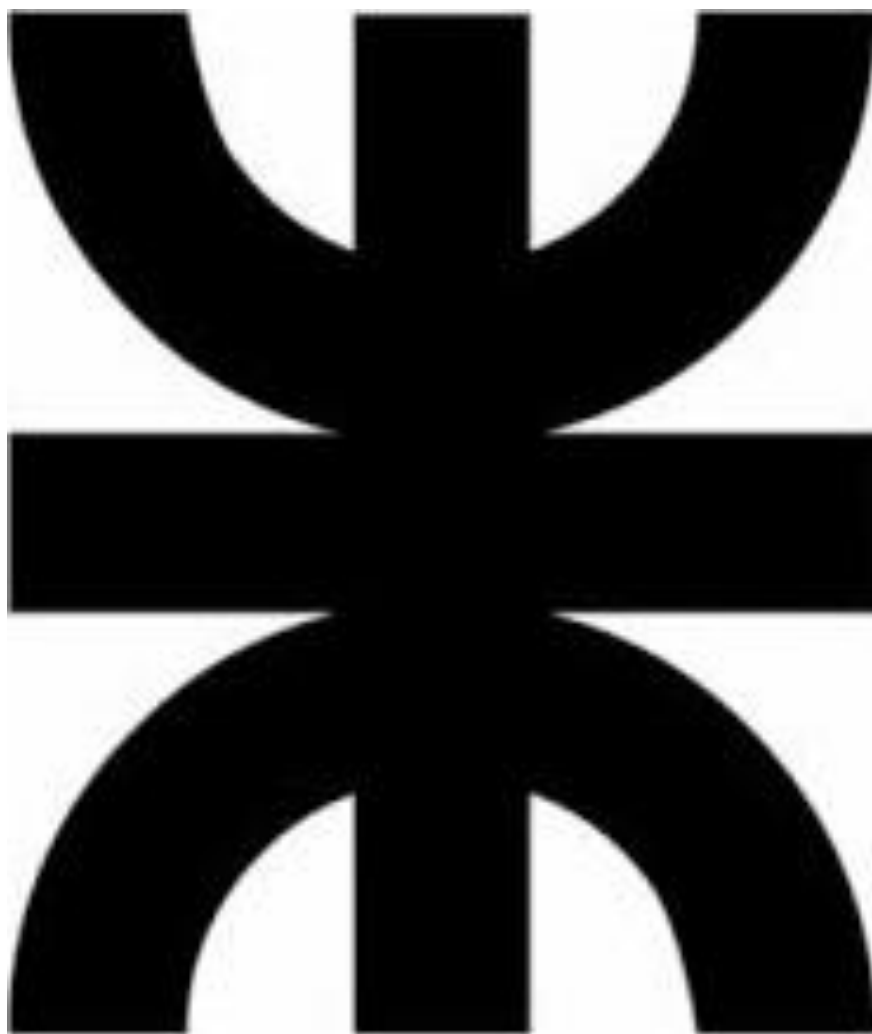




UTN – FRSF
ANÁLISIS NUMÉRICO
2026

INTEGRANTES:

MAURICIO ACOSTA

FEDERICO BLEUEL

MARCOS BUDE

PABLO DELLACQUA

Índice para trabajo práctico 4, trayectoria de dron:	
Introducción:	3
1. Extracción por Flujo Óptico y Diagnóstico Cinemático	3
Vínculo teórico con Interpolación Numérica:	4
Justificación de los parámetros computacionales:	5
Diagrama de Bloques	5
Rapidez instantánea	5
2. Modelado de Trayectoria	6
Método global	6
Método local	6
Conclusión	7
3. Síntesis y Renderizado de Vuelo Filtrado	8
Caso A	8
Caso B	10
Conclusión:	11
4. Modelado de Condiciones Reales de Vuelo	11
Velocidad y Aceleración	11
Implementación de Parámetros	12
Restricción en Curvas	12
Calibración Numérica (Desacople Cinemático)	13
Resultados del Flujo Óptico	13
Conclusión	13
Conclusión final	14
Referencias	15

Introducción:

En la imagen satelital proporcionada (mapa_satelital_completo.jpg) se observa el campus de la Universidad Tecnológica Nacional - Facultad Regional Santa Fe y sus alrededores. Un piloto de dron intenta realizar un mapeo fotogramétrico automático de la zona siguiendo una ruta de inspección programada. La aeronave despegue y debe aterrizar exactamente en el mismo punto de control inicial.

Sin embargo, debido a interferencias de vibración en el gimbal de la cámara, ráfagas de viento e imperfecciones en el bucle de control, la aeronave presenta severas fluctuaciones en su velocidad e incertidumbre en la altitud de vuelo. El resultado directo de esta captura visual se encuentra grabado en el archivo vuelo_dron.mp4.

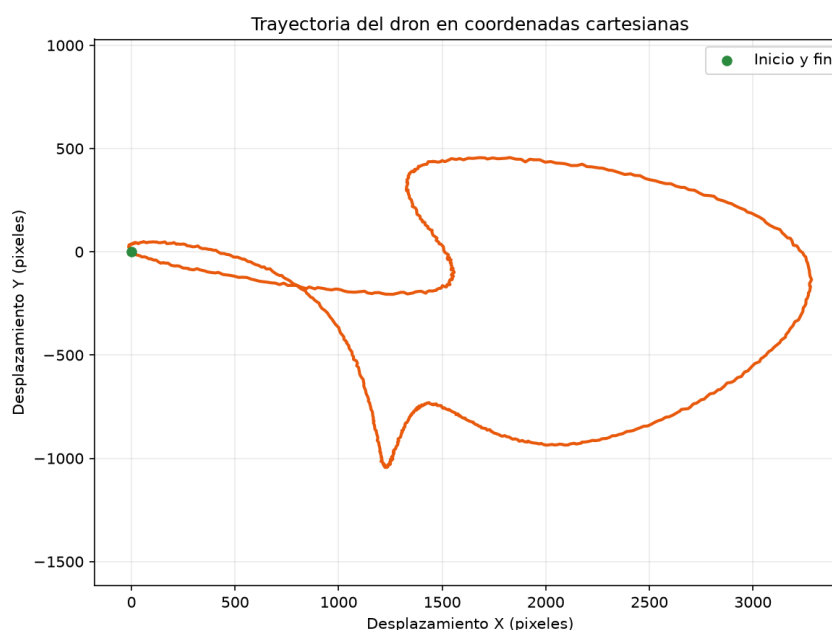
El objetivo como equipo de ingeniería es procesar la información visual del video, diagnosticar las fallas cinemáticas, modelar la trayectoria real mediante métodos de interpolación numérica y re-sintetizar el vuelo optimizado.

La resolución de este trabajo práctico se encuentra en el repositorio de Git:

<https://github.com/pabloDLQ/Analisis-numerico-2026>.

1. Extracción por Flujo Óptico y Diagnóstico Cinemático

El algoritmo de Lucas-Kanade resuelve la extracción de trayectoria asumiendo que el desplazamiento entre imágenes consecutivas es lo suficientemente pequeño como para aproximarlos con un desarrollo de Taylor de primer orden.



Para determinar las variaciones de altitud del dron, el algoritmo no mide la altura directamente, sino que la infiere a partir del zoom relativo de la escena (expansión/contracción visual).

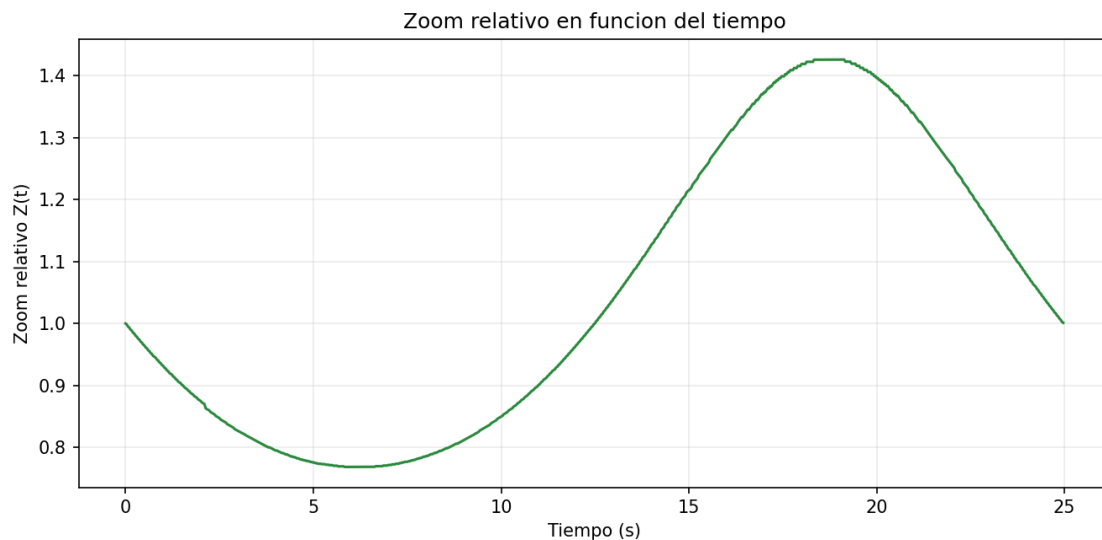
Se calcula el centro geométrico de todos los puntos válidos rastreados por el flujo óptico entre cada par de frames consecutivos.

Respecto a este centro se mide la distancia radial de cada punto, tanto en la imagen anterior como en la actual.

Al calcular el cociente entre las distancias nuevas y las antiguas, se obtiene un factor de escala. Para que este valor sea robusto y consistente frente a puntos mal emparejados, se toma la mediana estadística de todas las proporciones.

Este factor de zoom se multiplica y acumula a medida que avanza el video para tener la escala global respecto al punto de despegue.

Debido a la perspectiva de la cámara, el tamaño aparente de la imagen es inversamente proporcional a la distancia del suelo. Por lo tanto, la altitud relativa $Z(t)$ se obtiene calculando la inversa matemática del zoom acumulando $(1/\text{zoom})$.



Vínculo teórico con Interpolación Numérica:

Al reemplazar la aproximación de Taylor en la función de error, el problema de tracking abandona la fuerza bruta y se convierte en un sistema lineal sobredeterminado $Au = b$.

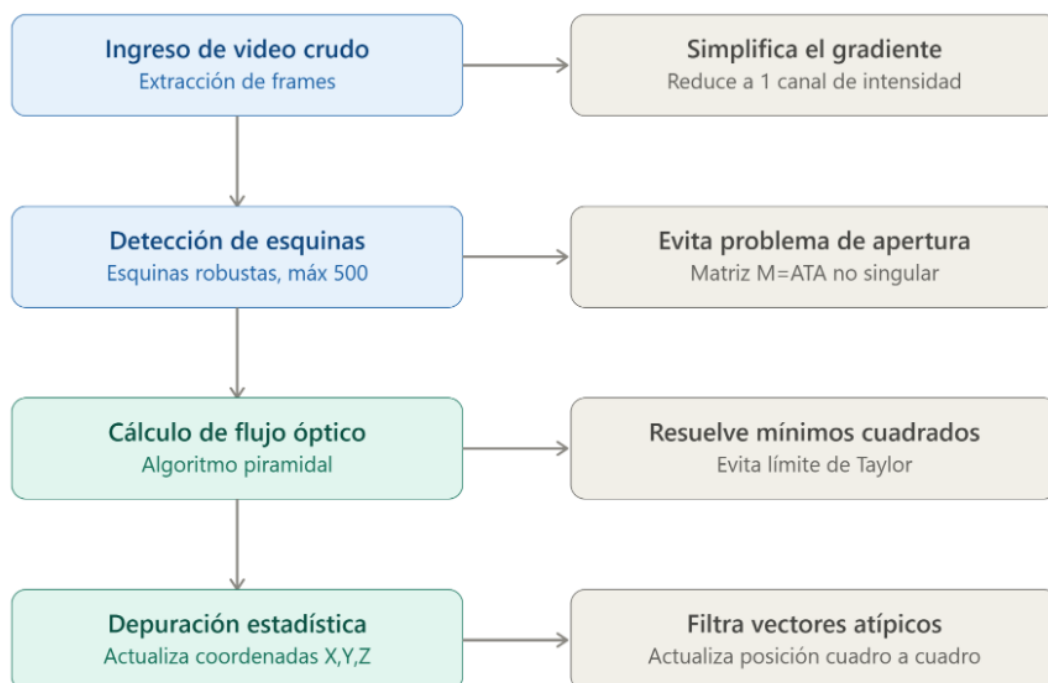
La solución explícita que minimiza este error es $u = (A^T A)^{-1} A^T b$.

Este enfoque es exactamente el método de mínimos cuadrados resuelto mediante ecuaciones normales, siendo el mismo fundamento analítico que sustenta el ajuste polinomial global como se indica en el Anexo A.

Justificación de los parámetros computacionales:

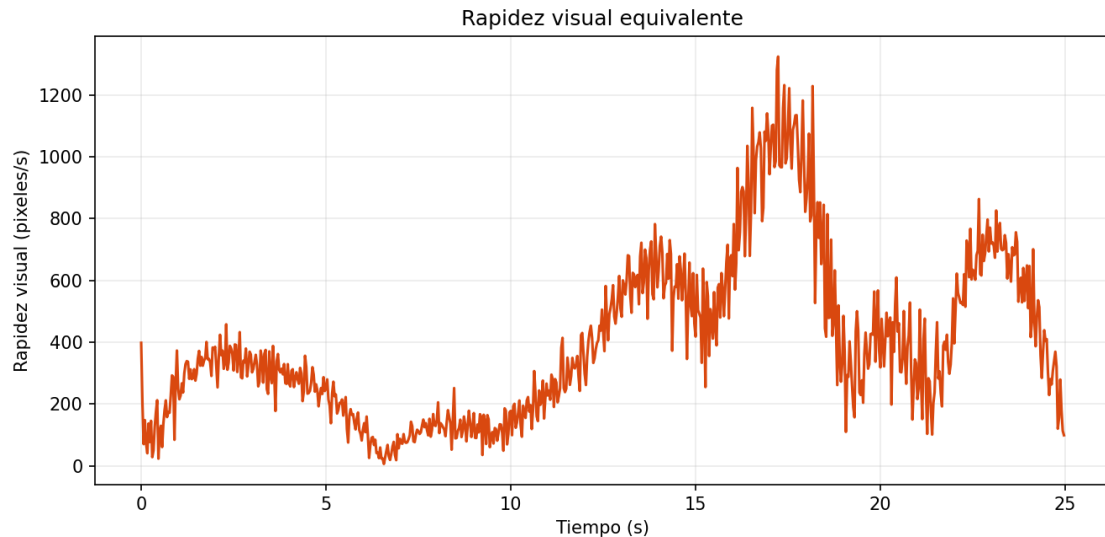
- Pre-selección de puntos: Para que la matriz del sistema (el tensor de estructura $M=ATA$) no sea singular, se deben descartar zonas planas o líneas continuas que generan el problema de apertura. El script emplea detectores para priorizar esquinas bien definidas, configurando un máximo de 500 características a rastrear.
- Tamaño de ventana (winSize): Fijado en (21, 21) píxeles, funciona como un hiperparámetro de compromiso. Si la ventana crece, el sistema recibe más ecuaciones ganando robustez frente al ruido de los sensores. Sin embargo, aumenta el riesgo físico de que los píxeles dentro del parche no se muevan de forma uniforme.
- Pirámide de resolución (maxLevel): Configurado con un nivel de profundidad de 3. Esta pirámide permite evadir la limitación matemática del desarrollo de Taylor (desplazamientos estrictamente pequeños) estimando movimientos bruscos en versiones reducidas del parche para luego refinar el cálculo progresivamente en su tamaño real.

Diagrama de Bloques



Rapidez instantánea

La obtención de la rapidez instantánea requiere calcular una derivada numérica sobre los datos de posición, este es un proceso que amplifica las fluctuaciones aleatorias.



Al medir variaciones minúsculas (ruido) entre intervalos de tiempo muy cortos, la diferencia finita convierte esas perturbaciones espaciales en picos de velocidad.

2. Modelado de Trayectoria

Para comenzar el modelado el algoritmo no utiliza los frames ruidosos, sino que submuestra la señal tomando un nodo representativo cada 30 muestras temporales. Esto define los puntos de anclaje (nodos) sobre los cuales trabajarán los métodos de interpolación.

Método global

Se implementó una función que te estima una función polinómica que se aproxime a los nodos seleccionados que sea del grado que se le pasa como argumento, con esa función probamos distintos grados de polinomio y encontramos que el que mejor se ajusta a la trayectoria es el de grado 16.

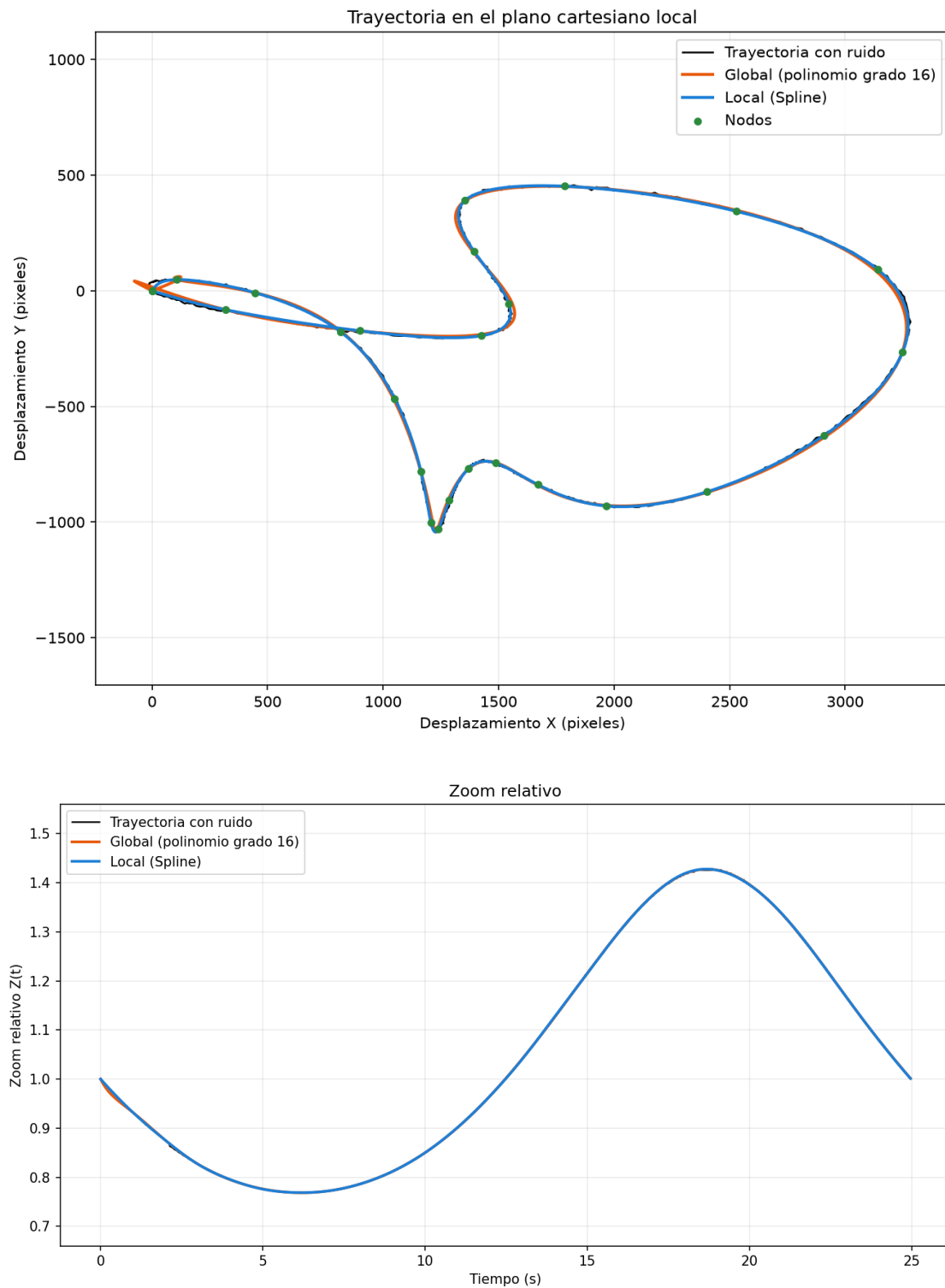
Dado que se exige que el dron aterrice exactamente sobre el punto de partida, el script implementa una corrección analítica. Calcula el error de ajuste en el primer y último nodo y aplica una compensación lineal fraccional a lo largo del tiempo para forzar que el inicio y el fin coincidan.

Método local

Se desarrolló un método de interpolación por tramos utilizando funciones cúbicas. Se impone la condición de curvatura nula (derivada segunda igual a cero) en el primer y último nodo del vuelo.

Se plantea un sistema de ecuaciones matricial tridiagonal, donde las incógnitas son las derivadas segundas en cada nodo interior, permitiendo luego reconstruir la posición del dron muestra a muestra.

Conclusión



El polinomio global es rígido, intentar que una única función pase cerca de todos los nodos del vuelo suele generar sobretiros y oscilaciones severas en los tramos intermedios y extremos, si bien nosotros encontramos un polinomio de grado 16 que se adaptó muy bien al recorrido. Con respecto a esto nosotros encontramos algo llamado el fenómeno de Runge, que es la oscilación grande

que aparece cerca de los extremos de un intervalo cuando interpolas una función con un polinomio de grado alto usando nodos equiespaciados. Un cambio brusco en la trayectoria del dron afecta a toda la curva global. Si bien esta aplicación del polinomio cumple muy bien el recorrido del dron, en los otros grados probados, las oscilaciones eran mucho más fuertes, pero se puede apreciar este efecto igualmente en el inicio/final.

El Spline Cúbico Natural es superior para este caso. Al ser local, una ráfaga de viento o un giro cerrado en una esquina del mapa solo afecta a ese tramo específico. Además, los polinomios cúbicos garantizan que la trayectoria final tenga posición, velocidad y aceleración continuas, lo que elimina el ruido del video original y genera un modelado más estable para el vuelo del dron.

3. Síntesis y Renderizado de Vuelo Filtrado

Comenzaremos aclarando que seleccionamos el método local por tramos (Spline Cúbico Natural) debido a su superioridad para el modelado físico. El spline evita oscilaciones y el fenómeno de Runge propios del polinomio global, garantizando que la posición reconstruida tenga velocidad y aceleración continuas. Esta suavidad es indispensable para que el renderizado del dron virtual fluya sin sobresaltos ni cambios de dirección discontinuos que arruinarían el análisis de flujo óptico posterior.

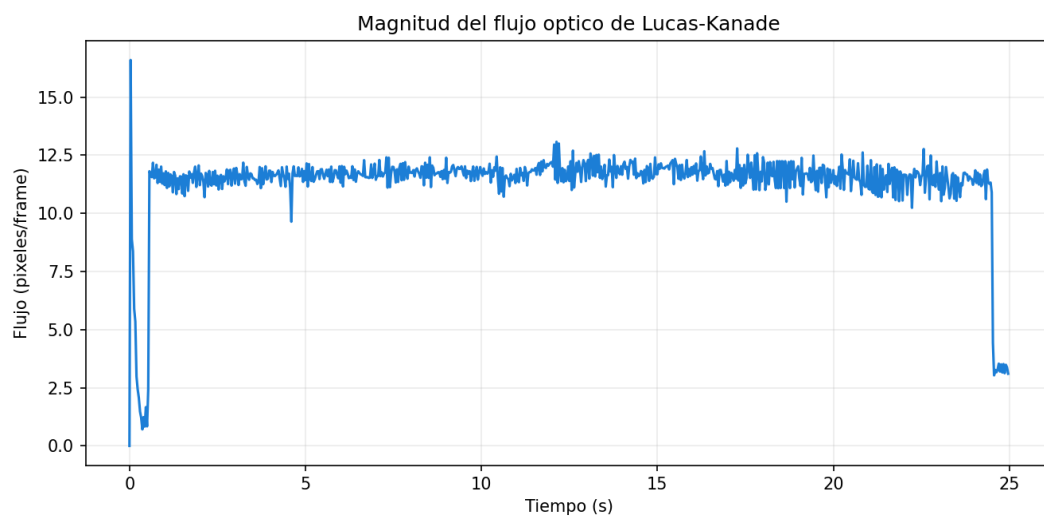
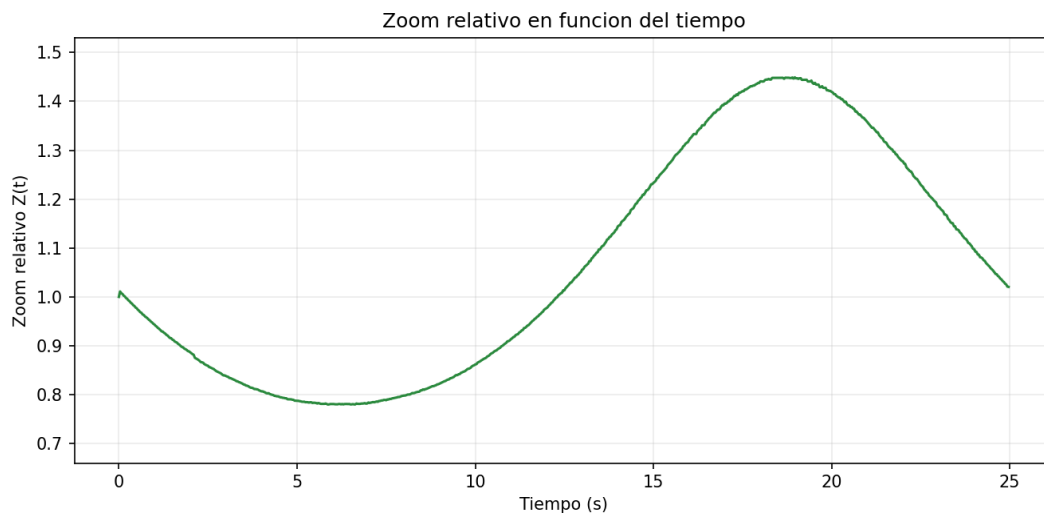
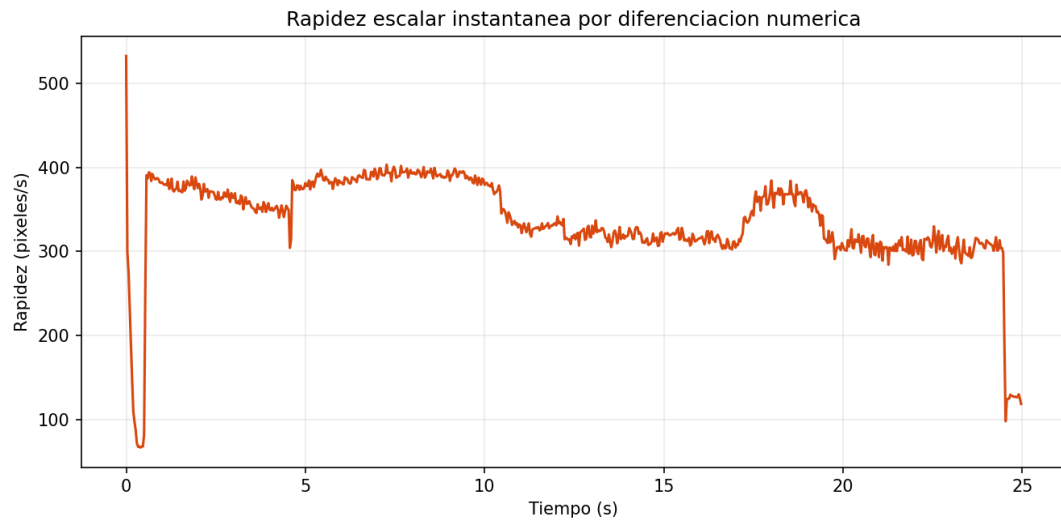
El algoritmo toma la trayectoria suavizada por el spline y la muestra densamente para medir con alta precisión la longitud de arco real (distancia recorrida acumulada) evaluando la hipotenusa entre saltos diferenciales de (X, Y).

Luego, en el caso B, se reparametriza la trayectoria para mantener aproximadamente constante la rapidez espacial, de modo que la distancia recorrida entre frames consecutivos sea prácticamente igual, mientras que en el caso A, se busca mantener constante la rapidez aparente en pantalla compensando las variaciones del zoom, esto es porque al aplicarle zoom al video, la velocidad que es constante se pierde, entonces tuvimos que compensarla para que

Caso A

Mantiene la información del zoom relativo original $Z(t)$, por lo que la cámara escala dinámicamente el recorte del mapa en función de la altura real construida.

Resultados:



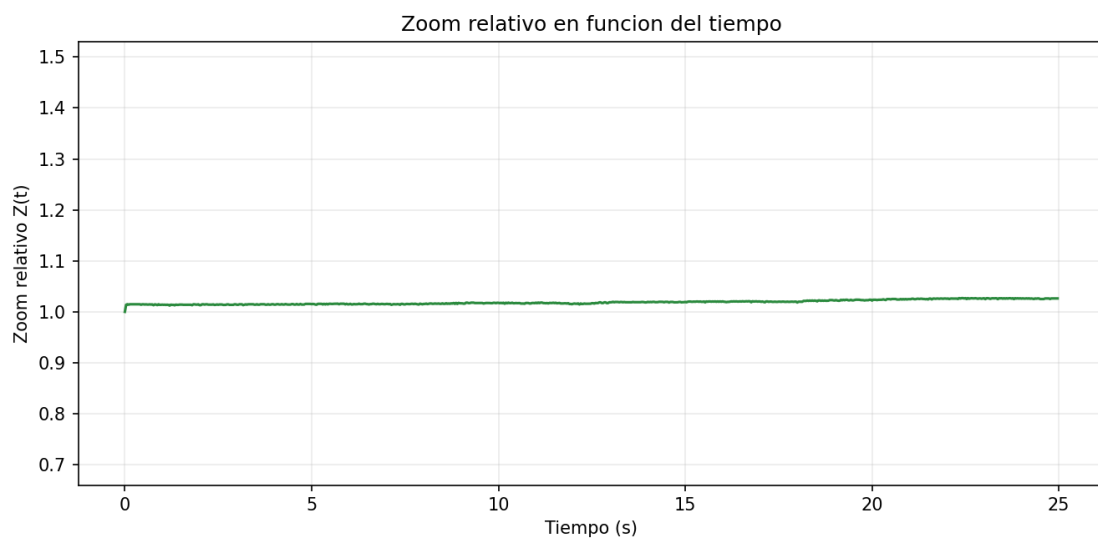
Con respecto a este video, nuestra mayor dificultad fue intentar mantener constante la velocidad, pero cuando se le aplica un zoom que no es constante,

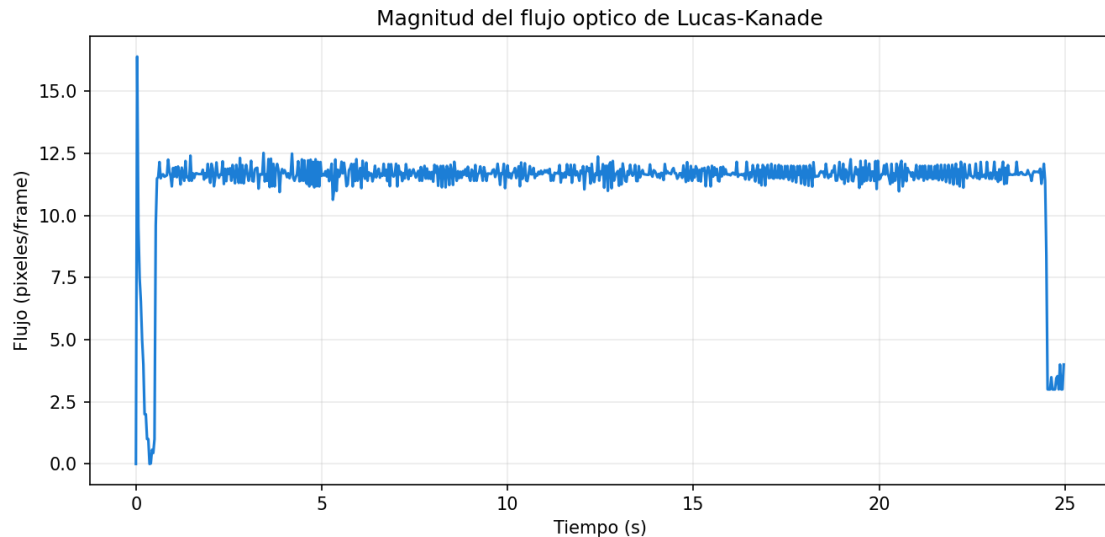
siempre nos encontramos problemas para que se mantenga así, entonces aplicamos algún tipo de corrección para el zoom, pero aun así tuvimos dificultades para mantenerla constante correctamente.

Caso B

Fija el factor de zoom en una constante de $Z=1.0$ durante toda la línea de tiempo, eliminando las perturbaciones de altitud.

Resultados:





Conclusión:

La física dicta que la aceleración lateral al tomar una curva es $a_c = \frac{v^2}{r}$. Al forzar que la rapidez escalar $v = v_0$ incluso al girar en curvas cerradas (donde el radio r tiende a cero), la aceleración requerida tiende a infinito, ese es el motivo de algunas de los picos que se aprecian en las funciones de velocidad.

En el mundo real, para generar esa fuerza lateral, un dron tendría que alcanzar un ángulo de inclinación irrealizable, el dron perdería estabilidad vertical y se estrellaría.

Al no frenar en las curvas, el desplazamiento en píxeles de la imagen satelital entre frames contiguos durante un giro brusco será tan grande que violará la hipótesis central de Lucas-Kanade (desplazamientos “suficientemente pequeños”). Esto causará que el flujo óptico pierda el rastro en los vértices más agudos de la trayectoria, demostrando por qué una rapidez estrictamente constante es incompatible con misiones de fotogrametría automáticas que tienen cambios de dirección, esto explicaría el fuerte ruido que hay en la gráfica de flujo óptico.

4. Modelado de Condiciones Reales de Vuelo

Al investigar un poco sobre sistemas de control de autopilotos para drones comerciales algunos sistemas y marcas más recomendados con los que nos topamos fueron ArduPilot y PX4, los cuales son ecosistemas de código abierto, como también el software de DJI.

Velocidad y Aceleración

La información que extraemos de estas marcas es que la velocidad máxima (o crucero) que alcanza en el software ArduPilot oscila entre **5.0 m/s** y **8.0m/s**,

nuestra elección fue de 7.0 m/s. Para la aceleración máxima (o tangencial) encontramos que ArduPilot limita entre 2.0 m/s² y 4.0 m/s², mientras que PX4 ajusta a 2.0 m/s² o 3.0 m/s², nuestra elección fue de 3.0 m/s².

Estos datos son suficientes para realizar la simulación de condiciones reales de vuelo, otro dato que se proporciona es el Jerk (o tirón), esto representa la tasa de cambio de la aceleración, según investigamos es la tercera derivada de la posición respecto al tiempo, en un gráfico se ve como una curva suavizada o con “forma de S” la utilización de este. El código que debíamos implementar para simular el Jerk es muy costoso computacionalmente, por lo tanto, decidimos no hacerlo y utilizar la velocidad y aceleración conocidas para hacer la simulación ya que la diferencia visual al implementar o no el Jerk es muy pequeña, el resultado en el gráfico se ve como un “perfil trapezoidal”.

Para evitar que en la implementación se utilicen iteraciones complejas y resolver polinomios de tercer grado implementamos la Ecuación de Torricelli:

$$v_f = \sqrt{v_0^2 + 2 \cdot a \cdot \Delta s}$$

Al usar esta fórmula, solo necesitamos la aceleración constante (A_{MAX}) y la distancia entre los nodos del spline (ds).

Para aplicar esta aceleración constante y asegurar que el dron respete los límites en las curvas, aplicamos un algoritmo de doble barrido.

Implementación de Parámetros

Primer Barrido: El algoritmo recorre la trayectoria desde el inicio aplicando la aceleración tangencial máxima (A_{MAX}), incrementando la velocidad hasta saturar en el límite cinemático local (v_{limite}) impuesto por la curvatura. Esto genera las rampas de aceleración.

Segundo Barrido: Se recorre la trayectoria en sentido inverso para propagar las restricciones de velocidad hacia atrás. Esto calcula la desaceleración necesaria con A_{MAX} , garantizando que la aeronave anticipe el frenado y respete la velocidad máxima exigida al ingresar a cada curva.

Restricción en Curvas

Para que el límite cinemático local mencionado en los barridos sea físicamente posible, se debió modelar la dinámica de giro del dron. En los multicopteros, el giro depende del ángulo máximo de alabeo (roll) permitido por el chasis, típicamente limitado a 31.45° para no perder sustentación. Utilizando la aceleración de la gravedad (9.81 m/s²), la aceleración centrípeta máxima sostenible se calculó analíticamente como $A_c = g \cdot \tan(31.45^\circ) \approx 6.0 \text{ m/s}^2$.

Si una curva en el spline superaba esta curvatura, el algoritmo calculaba la velocidad máxima segura y forzaba el frenado.

Calibración Numérica (Desacople Cinemático)

Dado que el algoritmo de renderizado y flujo óptico trabaja en el espacio de la imagen, fue obligatorio establecer un factor de conversión entre la física real y los píxeles. Al aislar la cinemática en el Caso B del inciso anterior (sin el efecto distorsivo del zoom), la velocidad observada se estabilizó en 350 px/s. Al vincular este límite empírico con nuestra velocidad real de crucero elegida (7.0 m/s), se dedujo una escala cinemática exacta de 50 px/m (350/7). Este factor permitió mapear nuestros límites físicos ($A_{max} = 3.0 \text{ m/s}^2$ y $A_{c,max} = 6 \text{ m/s}^2$) directamente al código como 150 px/s² y 300 px/s² respectivamente, resolviendo la ambigüedad de escala del video original.

Resultados del Flujo Óptico

Finalmente, al someter el video sintetizado con estas condiciones reales al algoritmo de Lucas-Kanade, las gráficas resultantes muestran que el nuevo modelo utilizado es muy satisfactorio. A diferencia de los incisos previos, la rapidez escalar instantánea dejó de ser constante, se muestran rampas de aceleración limpias y frenadas pronunciadas en coincidencia exacta con los vértices más cerrados de la trayectoria. Se logró un vuelo matemáticamente libre del ruido de alta frecuencia del gimbal original, pero con un comportamiento dinámico muy cercano al de un autopiloto comercial.

Conclusión

En conclusión, logramos simular un vuelo físicamente realista aplicando parámetros de drones comerciales. Al definir la escala en 50 px/m, pudimos trasladar estas limitaciones al código de forma directa. Para mantener la eficiencia del sistema, usamos un algoritmo de doble barrido con la ecuación de Torricelli; esto nos permitió simular el frenado en las curvas respetando los límites de aceleración, sin la necesidad de calcular el Jerk. Finalmente, las gráficas de Lucas-Kanade confirmaron que el modelo funciona tal como esperábamos, el dron acelera de forma limpia y frena justo a tiempo al girar, logrando el perfil trapezoidal que comentamos anteriormente.

Conclusión final

Pudimos analizar el video original, extrayendo la información relevante para lograr reconstruir la trayectoria del dron combinando flujo óptico, resuelto por mínimos cuadrados, con interpolación numérica. El spline cúbico natural resultó mejor que el polinomio global: evita el fenómeno de Runge y da continuidad en posición, velocidad y aceleración. Forzar velocidad constante en las curvas generó aceleraciones centrípetas irreales, por lo que se agregaron restricciones físicas de curvatura, aceleración máxima y barrido forward/backward para obtener una trayectoria realista.

Referencias

Referencias técnicas consultadas:

- Documentación de PX4 (Jerk-limited Type Trajectory): Se detalla cómo el autopiloto genera las curvas suaves de aceleración para evitar saturar el controlador.
https://docs.px4.io/main/en/config_mc/mc_jerk_limited_type_trajectory
- Documentación de ArduPilot (Waypoint Navigation): Parámetros reales configurables (WPNAV_SPEED, WPNAV_ACCEL) que determinan el comportamiento en el frenado y corte de curvas.
<https://ardupilot.org/copter/docs/parameters.html#wpnav-parameters>